Homework 3: Attention Mechanisms

BAC ML

Topic: Attention in Transformer

Name: _____ **Date:** _____

Instructions. Show all work and intermediate computations for full credit. Unless otherwise stated, assume all matrices are initialized as given and *do not* apply any learned biases. Write final matrix/vector answers in clearly labeled form. Partial credit is awarded for correct reasoning even with arithmetic errors.

Reference Formulas

Throughout this assignment we use the **column-input convention**: each token embedding is a *column* of $X \in \mathbb{R}^{d_{\text{model}} \times n}$ (d_{model} rows, n token columns). The projection matrices act from the left via their transposes:

$$Q = (W^Q)^\top X, \qquad K = (W^K)^\top X, \qquad V = (W^V)^\top X,$$

where $W^Q, W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, giving $Q, K \in \mathbb{R}^{d_k \times n}$ and $V \in \mathbb{R}^{d_v \times n}$.

The scaled dot-product self-attention output is:

$$\text{Attn}(Q, K, V) \;=\; V \cdot \text{softmax}\left(\frac{K^\top Q}{\sqrt{d_k}}\right),$$

where softmax is applied *column-wise*: each column of $K^\top Q \in \mathbb{R}^{n \times n}$ is a vector of scores over all n keys for one query, and is normalised independently.

Problems

Problem 1. [10 pts] What Is Attention? A Conceptual Warm-Up.

Consider the following sentence and its Telugu translation:

English: *"Please pay attention when learning about attention or you will get a strike!"*

Telugu (transliteration):

"Dayacēsi aṭenṣan guriñci nērcukunēṭappūdu śraddha vahincaṇḍi, lēkapōtē mīku strike vastuṃdi!"

(If viewing on a Unicode-capable device, the original script reads: `dayachEsi aTenshan gurinchi nErchukUnETappuDu Sraddha vahinchanDi, lEkapOtE meeku strike vastuMdi!`)

When a human translator reads the Telugu sentence and produces the English word by word, they do not treat every source token equally at every step—some tokens "draw attention" more than others depending on what is currently being generated. Notice also how the threat of a "strike" at the end of the sentence might change which words feel most important! This is the core intuition behind the transformer attention mechanism.

(a) **The analogy.** Using the translation task above as your motivating example, explain in 3–5 sentences what the attention mechanism is doing conceptually. Your answer should address *why* not all tokens should contribute equally to a given output position, and *how* the model decides which tokens are most relevant.

(b) **The Query.** In the translation analogy, the *query* represents the current output position being generated. In one or two sentences, explain what a query "asks for" and why projecting the input into a dedicated query space is preferable to using the raw embedding directly.

(c) **The Key.** Explain what the *key* of a token represents in the same analogy. How does the dot product between a query and a key capture relevance?

(d) **The Value.** Once the model has assigned attention weights (via softmax), it aggregates the *values* of the attended tokens. Explain in one or two sentences what the value represents and why it is useful to have a separate value projection rather than re-using the key.

(e) **Putting it together.** The word "attention" appears *twice* in the English sentence (once as the English content word "attention," once as a transliterated loanword in the Telugu source). If a transformer were translating the Telugu sentence token by token, would you expect the two occurrences to produce *similar* or *different*

a. The attention goes through each of the words. For each word, the mechanism determines which previous word (or could be future word) is important to it. Not all words equally contribute because they are not all important. Importance is determined the dot product of what the current word asks and what other words offer.

b. The query asks for information that helps define the current word, such as modifiers. It is preferable to project this into a query space instead of using the raw embedding because it generalizes the point of the query.

c. The key is information a word contains which the query looks for. Its dot product with the query vector captures relevance as less relevant key-query pairs would be more orthogonal with smaller dot products compared to more relevant key-query pairs.

d. The value represents what the key provides to the query. It is important to use a value as it provides contributing information.

e. They should have different attention weight patterns. As they were translated, each attention had different contexts.

attention weight patterns over the Telugu source tokens? Justify your answer in 2–3 sentences.

Problem 2. [10 pts] Hand-Computing Scaled Dot-Product Attention.

Let $n = 2$, $d_k = d_v = 2$. You are given (each *column* is one token):

$$Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad V = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}.$$

(a) Compute the raw attention score matrix $S = K^\top Q$. State its dimension and explain what the (i, j) entry represents.

(b) Scale S by $\sqrt{d_k}$ to obtain $\hat{S}$.

(c) Apply softmax *column-wise* to $\hat{S}$ to obtain the attention weight matrix A. (You may leave answers as exact fractions involving e, or approximate to 4 decimal places.)

(d) Compute the attention output $Z = VA$. State its dimension.

(e) Interpret the first column of A: which token does query 1 attend to more strongly?

Problem 3. [6 pts] Softmax and Attention Weights.

(a) Let $\mathbf{s} = (2,\ 1,\ 0)^\top$. Compute $\text{softmax}(\mathbf{s})$ exactly and verify the weights sum to 1.

(b) Now let $\mathbf{s}' = (4,\ 2,\ 0)^\top$ (each entry scaled by 2). Compute $\text{softmax}(\mathbf{s}')$. How does the distribution change compared to part (a)?

(c) What does this reveal about the effect of large raw attention scores on gradient flow during training?

2

a. $S = K^T Q = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}^T \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$; 2×2

Entry (i,j) represents the relevance between the i-th key and j-th query

b. $\hat{S} = \frac{S}{\sqrt{d_k}} = \begin{bmatrix} 1/\sqrt{2} & 0 \\ 1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}$

c. $A = \text{Softmax}(\hat{S}) = \begin{bmatrix} e^{1/\sqrt{2}}/(2e^{1/\sqrt{2}}) & e^0/(e^0+e^{1/\sqrt{2}}) \\ e^{1/\sqrt{2}}/(2e^{1/\sqrt{2}}) & e^{1/\sqrt{2}}/(e^0+e^{1/\sqrt{2}}) \end{bmatrix} = \begin{bmatrix} 1/2 & 1/(1+e^{1/\sqrt{2}}) \\ 1/2 & e^{1/\sqrt{2}}/(1+e^{1/\sqrt{2}}) \end{bmatrix} = \begin{bmatrix} 0.5 & 0.3302 \\ 0.5 & 0.6698 \end{bmatrix}$

d. $Z = VA = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 0.5 & 0.3302 \\ 0.5 & 0.6698 \end{bmatrix} = \begin{bmatrix} 1 & 0.6604 \\ 1.5 & 2.0094 \end{bmatrix}$; 2×2

e. Query 1 attends to token 1 and 2 equally

3.

a. $\text{Softmax}(S) = \text{softmax}\begin{pmatrix} 2 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} e^2/(e^0+e^1+e^2) \\ e^1/(e^0+e^1+e^2) \\ e^0/(e^0+e^1+e^2) \end{pmatrix} = \begin{pmatrix} e^2/(1+e+e^2) \\ e/(1+e+e^2) \\ 1/(1+e+e^2) \end{pmatrix} = \begin{pmatrix} 0.67 \\ 0.24 \\ 0.09 \end{pmatrix}$; $0.09+0.24+0.67 = 1.00$

b. $\text{Softmax}(S) = \text{softmax}\begin{pmatrix} 4 \\ 2 \\ 0 \end{pmatrix} = \begin{pmatrix} e^4/(1+e^2+e^4) \\ e^2/(1+e^2+e^4) \\ 1/(1+e^2+e^4) \end{pmatrix} = \begin{pmatrix} 0.867 \\ 0.117 \\ 0.016 \end{pmatrix}$; $0.867+0.117+0.016 = 1.000$; less uniform

c. This reveals that large raw attention scores will grow way more than smaller ones, which will lead to disappearing gradients

Problem 4. [8 pts] Dimensionality Drill.

A transformer model has the following hyperparameters:

$$d_{\text{model}} = 512, \quad d_k = d_v = 64, \quad n = 6 \text{ tokens.}$$

(a) Under the column-input convention, state the dimensions of X, W^Q, and $Q = (W^Q)^\top X$.

(b) State the dimension of the score matrix $S = K^\top Q$.

(c) State the dimension of the attention weight matrix $A = \text{softmax}\big(S/\sqrt{d_k}\big)$, where softmax is applied column-wise.

(d) State the dimension of the attention output $Z = VA$. Why must its first dimension equal d_v?

Problem 5. [8 pts] Parameter Count.

A transformer encoder has the following configuration:

$$d_{\text{model}} = 256, \quad h = 4 \text{ heads}, \quad d_k = d_v = 64, \quad L = 6 \text{ layers.}$$

(a) With $W^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, how many parameters are in W^Q, W^K, W^V for a *single* head? How many total across all $h = 4$ heads?

(b) Including the output projection $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$, what is the total parameter count for the attention sub-layer in one layer?

(c) How many total attention parameters are there across all $L = 6$ layers?

(d) Suppose you double d_{model} to 512 (keeping $h = 4$, so $d_k = d_v = 128$). By what factor does the attention parameter count change?

4.

a. $X : 512 \times 6$; $W^Q : 512 \times 64$; $Q : 64 \times 6$

b. $S : 6 \times 6$

c. $A : 6 \times 6$

d. $Z : 64 \times 6$; Z is the weighted sum of V columns, keeping the same dimensions

5.

a. $W^Q = 256 \times 64 = 16{,}384$; Single head $= 3 \times 16{,}384 = 49{,}152$; 4 heads $= 49{,}152 \times 4 = 196{,}608$

b. $W^O = 4 \times 64 \times 256 = 65{,}536$; One layer: $196{,}608 + 65{,}536 = 262{,}144$

c. Total $= 262{,}144 \times 6 = 1{,}572{,}864$

d. By a factor of $2 \times 2 = 4$

Problem 6. [10 pts] Multi-Head Attention — Concepts & Intuition.

Background: What Is Multi-Head Attention?

So far in this course we have studied *single-head* self-attention, where one set of learned projections (W^Q, W^K, W^V) produces a single set of queries, keys, and values. The output is one weighted mixture of value vectors, shaped by a single attention distribution.

Multi-head attention (MHA) runs h independent attention "heads" in parallel. Each head i has its own projections (W_i^Q, W_i^K, W_i^V) and computes:

$$\text{head}_i = \text{Attn}\big((W_i^Q)^\top X,\ (W_i^K)^\top X,\ (W_i^V)^\top X\big).$$

The outputs of all h heads are *concatenated* along the feature dimension and projected back to d_{model} via an output matrix W^O:

$$\text{MHA}(X) = W^O \begin{bmatrix} \text{head}_1 \\ \vdots \\ \text{head}_h \end{bmatrix}.$$

The standard design choice sets $d_k = d_v = d_{\text{model}}/h$, so each head works in a lower-dimensional subspace. This keeps the total parameter count comparable to a single full-dimensional attention head while allowing the model to attend across *multiple independent representation subspaces simultaneously.*

Empirically, different heads specialise: one head might capture syntactic subject–verb dependencies, another coreference (which noun a pronoun refers to), and another positional proximity. A single head must average all these signals into one distribution and one output—inevitably losing information that competing patterns cannot share.

(a) **Benefits of multi-head attention.** In 4–6 sentences, explain why multi-head attention is preferred over single-head attention in practice. Address at least two distinct advantages, drawing on the intuition in the background above. (*Hint: consider what a single head sacrifices when it must encode multiple types of relationships simultaneously.*)

(b) **Head specialisation and redundancy.** Suppose you train a transformer with $h = 8$ heads on a translation task and, after inspecting the learned attention patterns, you discover that 6 of the 8 heads have converged to nearly identical attention distributions.

(i) What does this suggest about how the model is using its representational capacity? Is this necessarily a problem, or could there be a benign explanation?

(ii) Researchers have proposed *attention dropout*—randomly zeroing entire heads during training—as a regularisation technique. In 2–3 sentences, explain how this might encourage greater diversity across heads and reduce redundancy.

Good luck! Remember: understanding the shapes is half the battle.

a. MHA provides more complex meaning. In SHA, one embedding leads to tradeoffs in representing different relationships at once. MHA allows multiple heads to focus on different relationships. Additionally, the lower dimensions of the MHA allows for efficient computations across parallel heads.

b.

i. The model is using its representative capacity on the same relationships. This is not that big of an issue, just inefficient.

ii. Attention dropout ensures that other heads cannot rely on a singular head to represent relationships if it has the possibility of being dropped. Then, more relationships will be represented as the model does not overfit to one head's representation.